



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ INŻYNIERII METALI I INFORMATYKI PRZEMYSŁOWEJ
KATEDRA INFORMATYKI STOSOWANEJ I MODELOWANIA

Sprawozdanie z projektu

Osobista aplikacja do zarządzania zadaniami oraz wydarzeniami

Autor:	Aleksander Buszek
Kierunek:	Informatyka Techniczna
Typ studiów:	stacjonarne II stopnia
Rok:	2025/2026
Semestr:	1
Grupa:	1

Kraków, 2026

Spis treści

1	Cel projektu	4
2	Użyte technologie	5
3	Zakładane funkcjonalności	6
4	Schemat aplikacji	7
5	Projekt bazy	8
6	Fragmenty kodu	10
6.1	Docker	10
6.2	Backend	11
6.2.1	Uwierzytelnianie JWT	11
6.2.2	Obsługa dodania wydarzenia/wydarzeń cyklicznych	11
6.2.3	Model wydarzenia	12
6.3	Frontend	13
6.3.1	Odświeżanie tokenu	13
6.3.2	Główny widok aplikacji	15
6.3.3	Definicja modalu	17
7	Wygląd aplikacji	19
7.1	Widok logowania/rejestracji	19
7.2	Widok główny	20
7.3	Modale	21
7.3.1	DayEventsModal	21
7.3.2	EventDetailsModal	21
7.3.3	ConfirmDeleteModal	23
7.3.4	AddEventModal	24
7.3.5	Edycja istniejącego wydarzenia	25
7.4	Informacje o użytkowniku	26
8	Testy	27
9	Wnioski	29

Rozdział 1

Cel projektu

Celem projektu było zaprojektowanie oraz implementacja nowoczesnej aplikacji webowej umożliwiającej zarządzanie zadaniami oraz wydarzeniami w sposób intuicyjny i przejrzysty dla użytkownika. Aplikacja miała wspierać codzienną organizację czasu poprzez umożliwienie tworzenia, edycji oraz filtrowania wydarzeń, w tym również wydarzeń cyklicznych.

Istotnym założeniem było stworzenie systemu, który zapewnia pełną obsługę użytkownika od procesu rejestracji i uwierzytelniania, aż po zarządzanie jego danymi oraz zasobami w bezpieczny sposób. W tym celu wykorzystano mechanizmy autoryzacji oparte na tokenach JWT oraz odpowiednią strukturę backendu.

Kolejnym celem było zaprojektowanie skalowalnej i modularnej architektury aplikacji, opartej na podziale na warstwę frontendową, backendową oraz bazę danych. Dzięki temu możliwy jest dalszy rozwój systemu oraz jego rozbudowa o nowe funkcjonalności bez konieczności istotnych zmian w istniejącej strukturze.

Projekt miał również na celu wykorzystanie współczesnych technologii webowych oraz dobrych praktyk inżynierii oprogramowania. W szczególności skupiono się na:

- zastosowaniu frameworka Django do budowy backendu wraz z wykorzystaniem ORM,
- wykorzystaniu Next.js do stworzenia dynamicznego i responsywnego interfejsu użytkownika,
- integracji z relacyjną bazą danych PostgreSQL,
- wykorzystaniu Dockera do zapewnienia spójności środowiska uruchomieniowego.

Dodatkowym celem projektu było zdobycie praktycznego doświadczenia w pracy z wyżej wymienionymi technologiami oraz zrozumienie zasad budowy pełnostackowych aplikacji webowych, obejmujących zarówno warstwę serwerową, jak i kliencką.

Projekt miał także uwzględniać możliwość dalszej rozbudowy, w szczególności w kierunku dodania funkcjonalności współdzielonych wydarzeń (grupowych) oraz rozszerzenia mechanizmów zarządzania użytkownikami i ich danymi.

Rozdział 2

Użyte technologie

Projekt można podzielić na 3 części pod względem użytej technologii: **baza danych**, **backend** oraz **frontend**.

Jako **bazę danych** użyłem **PostgreSQL**. Jest to relacyjna baza danych wykorzystana do przechowywania danych aplikacji, zapewniająca niezawodność oraz obsługę złożonych zapytań SQL. Została wybrana ze względu na bardzo dobrą integrację z frameworkiem Django.

Jako **backend** użyłem **Django**. Jest to framework webowy w języku Python wykorzystany do stworzenia warstwy backendowej aplikacji oraz obsługi logiki biznesowej. Został wybrany ze względu na wbudowany system ORM, mechanizmy uwierzytelniania oraz szybkie tworzenie aplikacji zgodnie z dobrymi praktykami. Ponadto zapewnia panel admina, który był jest pomocny zarówno na początku tworzenia aplikacji, jak i już w trakcie jej funkcjonowania.

Jako **frontend** użyłem **Next.js**. Jest to framework frontendowy oparty na **React**, wykorzystany do budowy interfejsu użytkownika aplikacji. Został wybrany ze względu na optymalizację wydajności oraz wygodne zarządzanie routingiem.

Do uruchamiania i zarządzania środowiskiem aplikacji wykorzystano **Docker**. Umożliwia on uruchamianie bazy danych, backendu i frontendu w odizolowanych kontenerach, co zapewnia spójność środowiska oraz ułatwia wdrożenie i rozwój aplikacji.

Rozdział 3

Zakładane funkcjonalności

Założono następujące funkcjonalności:

- rejestracja użytkowników
- autoryzacja i logowanie użytkowników
- dodawanie wydarzeń jednorazowych oraz cyklicznych
- filtrowanie wydarzeń po dacie, kategorii, etapie wykonania zadań

W trakcie tworzenia aplikacji powstały następujące funkcjonalności:

- dodawanie własnych kategorii wraz z identyfikującym je kolorem
- dodawanie własnych statusów, różnych od domyślnych
- wylogowywanie użytkownika
- edytowanie wydarzeń
- filtrowanie wydarzeń po statusie ich realizacji

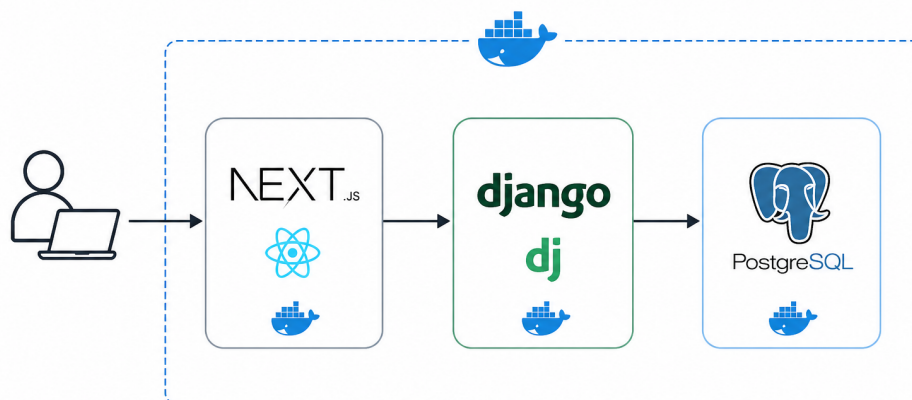
Dodatkową funkcjonalnością, która nie została uwzględniona w początkowych założeniach projektowych, był sposób przechowywania tokenów JWT po stronie klienta. Początkowo planowano wykorzystanie mechanizmu `localStorage`, jednak po analizie zagrożeń bezpieczeństwa związanych z tym podejściem, w szczególności podatności na ataki typu XSS (Cross-Site Scripting), zdecydowano się na zmianę tej koncepcji.

Ostatecznie tokeny JWT przechowywane są w **ciasteczkach oznaczonych jako `HttpOnly`**, co uniemożliwia dostęp do nich z poziomu kodu JavaScript wykonywanego w przeglądarce. Takie rozwiązanie znacząco zwiększa poziom bezpieczeństwa aplikacji, ograniczając ryzyko przejęcia sesji użytkownika przez złośliwe oprogramowanie.

Rozdział 4

Schemat aplikacji

Aplikacja działa w przeglądarce. Klient, za pośrednictwem przeglądarki, wchodzi w interakcje z aplikacją która je odbiera i przetwarza w części frontendowej w **Next.js**. Następnie są wysyłane zapytania do backendu w **Django** za pomocą odpowiednich endpointów w stylu architektonicznym **REST API**. Backend przetwarza zapytania, dokonuje autoryzacji i po poprawnym uwierzytelnieniu łączy się z bazą danych, **PostgreSQL**, na której są wykonywane operacje odczytu, zapisu czy modyfikacji odpowiednich rekordów.



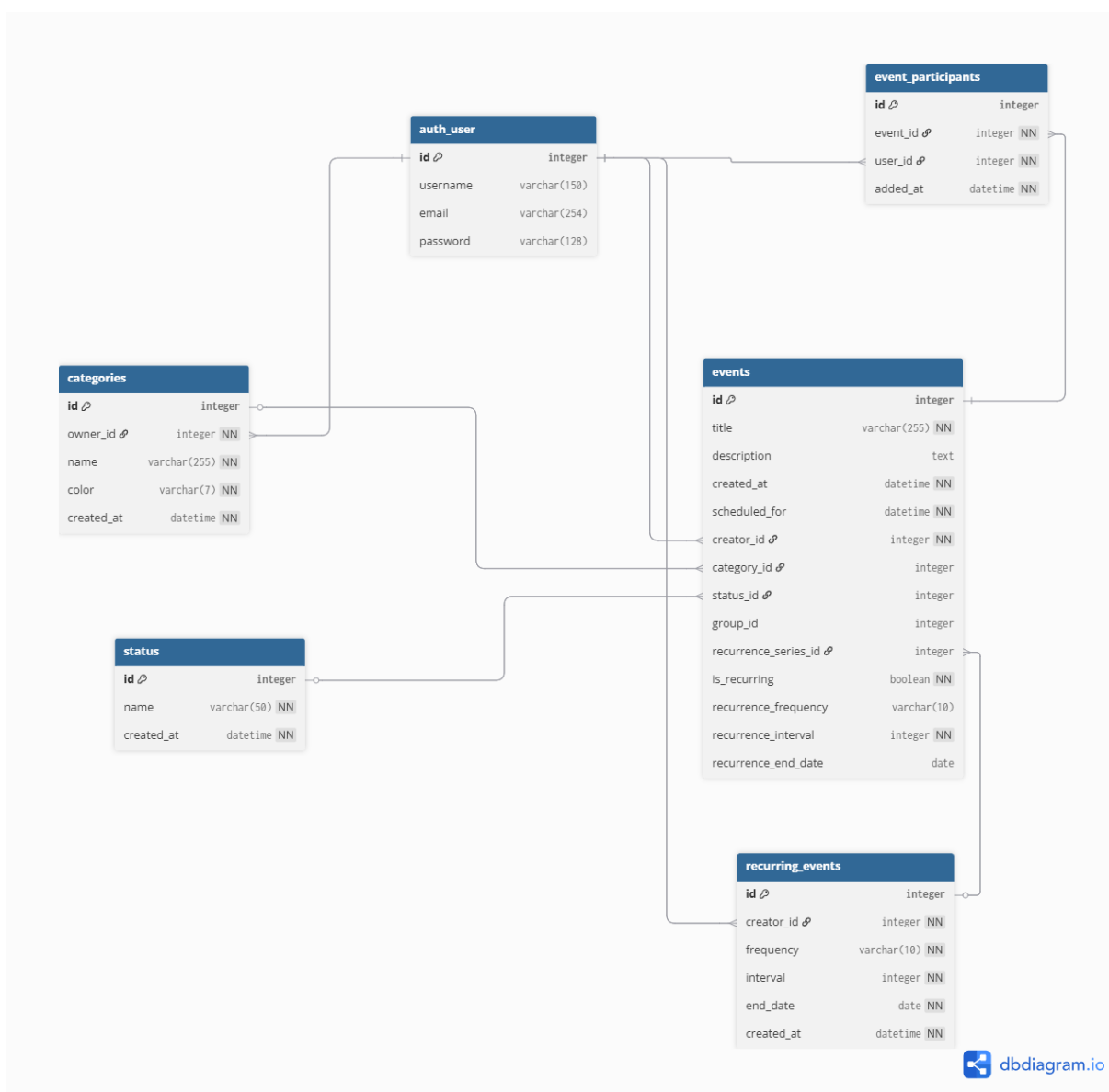
Rysunek 4.1: Schemat aplikacji

Całość jest skonteneryzowana w **Dockerze**, który zapewnia bezpieczne, izolowane środowisko uruchomieniowe i ułatwia dalszy deployment w celu na przykład hostowania aplikacji na zewnętrznym serwerze.

Rozdział 5

Projekt bazy

Schemat bazy danych, przechowujący dane aplikacji prezentuje się następująco:



Rysunek 5.1: Schemat bazy danych

Schemat posiada następujące tabele:

- **auth_user**- tabela przechowuje dane niezbędne do poprawnego uwierzytelniania użytkownika. Przechowuje nazwę użytkownika, email oraz zahashowane hasło.
- **event_participants**- tabela, która została umieszczona w celu prostszego, dalszego rozbudowania aplikacji w przyszłości o dodanie funkcjonalności wydarzeń grupowych gdzie do jednego wydarzenia może się zapisać wielu użytkowników
- **categories**- tabela przechowująca kategorie, które użytkownik może przypisać do jego wydarzeń. Każda kategoria ma dodatkowo przypisany swój kolor, w celu czytelniejszego wyświetlania i grupowania wizualnego wydarzeń w kalendarzu.
- **status**- tabela, dzięki której użytkownik może dodawać własne statusy, poza tymi domyślnymi dostarczonymi przez aplikację, czyli **Not Applicable, To Do, In Progress, Done**.
- **events**- tabela przechowująca dane o każdym pojedynczym wydarzeniu. Jest sercem całej aplikacji. Zawiera takie informacje jak swój opis, datę wydarzenia oraz wszystkie informacje o cykliczności danego wydarzenia.
- **recurring_events**- tabela przechowująca dane o cyklicznych wydarzeniach. Dzięki niej wiemy z jaką częstotliwością dzieją się dane wydarzenia i kiedy się kończą.

Tak zaprojektowany schemat, dzięki normalizacji, można w przyszłości dalej rozwijać.

Rozdział 6

Fragmenty kodu

6.1. Docker

W celu łatwego i powtarzalnego deploymentu aplikacji utworzyłem Dockerfile z instrukcjami budowania obrazu osobno dla frontendu i backendu. Te instrukcje są połączone w `docker-compose.yml`.

```
1 FROM python:3.13-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \
4     PYTHONUNBUFFERED=1
5
6 WORKDIR /app
7
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir -r requirements.txt
10
11 COPY backend_component/ .
12
13 EXPOSE 8000
14
15 CMD ["sh", "-c", "python manage.py migrate && python manage.py runserver
    0.0.0.0:8000"]
```

Listing 6.1: Dockerfile dla backendu

```
1 FROM node:24-slim
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm ci
7
8 COPY . .
9
10 EXPOSE 3000
11
12 CMD ["npm", "run", "dev", "--", "-H", "0.0.0.0"]
```

Listing 6.2: Dockerfile dla frontendu

6.2. Backend

6.2.1. Uwierzytelnianie JWT

Z racji, że zdecydowałem się na trzymanie tokenów w cookies, nie local storage, nie mogę pobierać tokenów z nagłówku **Authorization**. W tym celu zaimplementowałem własną klasę odpowiedzialną za uwierzytelnianie.

```

1 class CookieJWTAuthentication(JWTAuthentication):
2     def authenticate(self, request):
3         token = request.COOKIES.get('access_token')
4
5         if not token:
6             return None
7
8         try:
9             validated_token = self.get_validated_token(token)
10        except AuthenticationFailed as e:
11            raise AuthenticationFailed(f"Token validation failed:
12            {str(e)}")
13        try:
14            user = self.get_user(validated_token)
15            return user, validated_token
16        except AuthenticationFailed as e:
17            raise AuthenticationFailed(f"User retrieval failed: {str(e)}")

```

Listing 6.3: Uwierzytelnianie JWT

6.2.2. Obsługa dodania wydarzenia/wydarzeń cyklicznych

Wydarzenia cykliczne dodajemy za pomocą tej samej funkcji co zwykle. Do decydowania, czy wydarzenie jest cykliczne czy nie służy zmienna `is_recurring`. Proces dodawania wygląda następująco:

```

1 @transaction.atomic
2 def perform_create(self, serializer):
3     event = serializer.save(creator_id=self.request.user)
4     EventParticipants.objects.create(event_id=event,
5     user_id=self.request.user)
6
7     if not event.is_recurring:
8         return
9
10    # For recurring events, we create the entire series upfront to
11    simplify retrieval and management.
12    recurrence_series = RecurringEvents.objects.create(
13        creator_id=self.request.user,
14        frequency=event.recurrence_frequency,
15        interval=event.recurrence_interval,
16        end_date=event.recurrence_end_date,
17    )
18    event.recurrence_series_id = recurrence_series
19    event.save(update_fields=['recurrence_series_id'])
20
21    occurrence = add_recurrence_interval(
22        event.scheduled_for,

```

```

20     recurrence_series.frequency,
21     recurrence_series.interval,
22 )
23 while occurrence.date() <= recurrence_series.end_date:
24     next_event = Events.objects.create(
25         title=event.title,
26         description=event.description,
27         scheduled_for=occurrence,
28         creator_id=self.request.user,
29         category_id=event.category_id,
30         status_id=event.status_id,
31         group_id=event.group_id,
32         recurrence_series_id=recurrence_series,
33         is_recurring=True,
34         recurrence_frequency=recurrence_series.frequency,
35         recurrence_interval=recurrence_series.interval,
36         recurrence_end_date=recurrence_series.end_date,
37     )
38     EventParticipants.objects.create(event_id=next_event,
39                                     user_id=self.request.user)
40
41     next_occurrence = add_recurrence_interval(
42         occurrence,
43         recurrence_series.frequency,
44         recurrence_series.interval,
45     )
46     if next_occurrence <= occurrence:
47         break
48     occurrence = next_occurrence

```

Listing 6.4: Dodawanie wydarzeń

Funkcja jest opakowana w dekorator `transaction.atomic`, dzięki czemu mamy pewność, że w przypadku wystąpienia błędu w dodawaniu wydarzeń, nastąpi rollback a baza danych zostanie w spójnym stanie.

6.2.3. Model wydarzenia

Framework Django udostępnia mechanizm ORM (Object-Relational Mapping), który pozwala definiować strukturę bazy danych w sposób obiektowy, zgodny ze składnią języka Python. Dzięki temu tworzenie tabel sprowadza się do definiowania klas i ich atrybutów, co znacząco upraszcza pracę z bazą danych.

Na listingu 6.5 przedstawiono definicję modelu `Events`, który odpowiada tabeli przechowującej informacje o wydarzeniach. Model ten zawiera zarówno pola opisujące podstawowe właściwości wydarzenia (takie jak tytuł, opis czy termin realizacji), jak i relacje do innych tabel, m.in. użytkownika, kategorii oraz statusu.

Dodatkowo w modelu zdefiniowano dwie klasy wewnętrzne typu `TextChoices`, które pełnią rolę enumeracji.

```

1 class Events(models.Model):
2     # Enum defining available statuses for an event (local to Events model
3     for clarity and encapsulation)
4     class EventStatus(models.TextChoices):
5         NOT_APPLICABLE = 'not_applicable', 'Not Applicable'
6         TODO = 'todo', 'To Do'

```

```

6     IN_PROGRESS = 'in_progress', 'In Progress'
7     DONE = 'done', 'Done'
8
9     # Enum defining recurrence frequency options used specifically by the
Events model
10    class RecurrenceFrequency(models.TextChoices):
11        DAILY = 'daily', 'Every day'
12        WEEKLY = 'weekly', 'Every week'
13        MONTHLY = 'monthly', 'Every month'
14        YEARLY = 'yearly', 'Every year'
15
16    title = models.CharField(max_length=255)
17    description = models.TextField(null=True, blank=True)
18    created_at = models.DateTimeField(auto_now_add=True)
19    scheduled_for = models.DateTimeField()
20    creator_id = models.ForeignKey(User, on_delete=models.CASCADE,
21    related_name='created_events')
22    category_id = models.ForeignKey(Categories, on_delete=models.SET_NULL,
23    null=True, blank=True, related_name='events')
24    status_id = models.ForeignKey(Status, on_delete=models.SET_NULL,
25    null=True, blank=True, related_name='events')
26    group_id = models.IntegerField(null=True, blank=True)
27    recurrence_series_id = models.ForeignKey(
28        RecurringEvents,
29        on_delete=models.CASCADE,
30        null=True,
31        blank=True,
32        related_name='events',
33    )
34    is_recurring = models.BooleanField(default=False)
35    recurrence_frequency = models.CharField(
36        max_length=10,
37        choices=RecurrenceFrequency.choices,
38        null=True,
39        blank=True,
40    )
41    recurrence_interval = models.PositiveIntegerField(default=1)
42    recurrence_end_date = models.DateField(null=True, blank=True)

```

Listing 6.5: Definicja tabeli Events

6.3. Frontend

6.3.1. Odświeżanie tokenu

Kod konfiguruje klienta HTTP przy użyciu biblioteki Axios do komunikacji z backendem oraz implementuje mechanizm automatycznego odświeżania tokena uwierzytelniającego. W przypadku otrzymania błędu 401 (Unauthorized) interceptor próbuje odnowić token, a następnie ponawia pierwotne zapytanie. Dodatkowo zastosowano mechanizm współdzielenia jednej obietnicy (Promise), aby uniknąć wielokrotnego wykonywania żądania odświeżania tokena równocześnie.

```

1 import axios, { AxiosError, InternalAxiosRequestConfig } from "axios";
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";

```

```

4
5 "This module sets up an Axios instance with a response interceptor to
  handle token refresh logic. When a 401 Unauthorized error is
  encountered, it attempts to refresh the access token using a single
  shared promise to prevent multiple simultaneous refresh requests. If
  the refresh is successful, it retries the original request. If the
  refresh fails or if the original request was for login or token refresh
  endpoints, it rejects the error."
6
7 type RetriableRequestConfig = InternalAxiosRequestConfig & {
8   _retry?: boolean;
9 };
10
11 let refreshPromise: Promise<unknown> | null = null;
12
13 const api = axios.create({
14   baseURL: API_URL,
15   withCredentials: true,
16 });
17
18 const refreshAccessToken = async () => {
19   if (!refreshPromise) {
20     refreshPromise = api
21       .post("/login/token/refresh/")
22       .finally(() => {
23         refreshPromise = null;
24       });
25   }
26
27   return refreshPromise;
28 };
29
30 api.interceptors.response.use(
31   (response) => response,
32   async (error: AxiosError) => {
33     const originalRequest = error.config as RetriableRequestConfig |
34     undefined;
35     const status = error.response?.status;
36     const requestUrl = originalRequest?.url ?? "";
37
38     if (!originalRequest || status !== 401 || originalRequest._retry) {
39       return Promise.reject(error);
40     }
41
42     if (requestUrl.includes("/login/login/") ||
43     requestUrl.includes("/login/token/refresh/")) {
44       return Promise.reject(error);
45     }
46
47     originalRequest._retry = true;
48
49     try {
50       await refreshAccessToken();
51       return api(originalRequest);
52     } catch (refreshError) {
53       return Promise.reject(refreshError);
54     }
55   },

```

```

54 );
55
56 export { API_URL, api };

```

Listing 6.6: Klient odświeżający token JWT

6.3.2. Główny widok aplikacji

Poniższy fragment przedstawia główny widok aplikacji odpowiedzialny za wyświetlanie kalendarza wydarzeń oraz obsługę interakcji użytkownika. Komponent zawiera przyciski nawigacyjne, panel filtrowania, widok kalendarza oraz zestaw okien modalnych służących do podglądu, dodawania i usuwania wydarzeń.

```

1  return (
2    <div className="min-h-screen bg-slate-900 text-white">
3      <div className="relative max-w-4xl mx-auto p-4">
4        <div className="mb-4 flex justify-end gap-3">
5          <Link
6            href="/user/info"
7            className="..."
8          >
9            User info
10         </Link>
11         <button
12           type="button"
13           onClick={handleLogout}
14           disabled={isLoggingOut}
15           className="..."
16         >
17           {isLoggingOut ? "Logging out..." : "Logout"}
18         </button>
19       </div>
20
21       {/* Main content area with calendar and filters */}
22       {/* Filter Panel */}
23       <EventFiltersPanel
24         categories={categories}
25         statuses={statuses}
26         selectedCategoryId={selectedCategoryId}
27         selectedStatusId={selectedStatusId}
28         filteredEventsCount={filteredEvents.length}
29         totalEventsCount={events.length}
30         onCategoryChange={setSelectedCategoryId}
31         onStatusChange={setSelectedStatusId}
32         onClearFilters={() => {
33           setSelectedCategoryId("all");
34           setSelectedStatusId("all");
35         }}
36       />
37
38       {/* Calendar View */}
39       <CalendarViewModel
40         events={filteredEvents}
41         categories={categories}
42         onDateClick={handleDateClick}
43         onMonthChange={(year, month) => {

```

```

44         setVisibleMonth((current) => {
45             if (current.year === year && current.month ===
month) {
46                 return current;
47             }
48             return { year, month };
49         });
50     }}
51 />
52
53 /* Modal for displaying events on a specific day */
54 {isModalOpen && selectedDate && (
55     <DayEventsModal
56         selectedDate={selectedDate}
57         events={selectedDayEvents}
58         onClose={() => {
59             setIsModalOpen(false)}}
60         onAddEvent={() => setAddEventModal(true)}
61         onSelectedEvent={(event) => {
62             setSelectedEvent(event);
63             setIsEventModalOpen(true);
64         }}
65     />
66 )}
67
68 /* Modal for adding new events */
69 {addEventModal && (
70     <AddEventModal
71         newEvent={newEvent}
72         categories={categories}
73         onChange={handleChangeNewEvent}
74         onClose={() => {
75             setAddEventModal(false);
76             setNewEvent({
77                 title: "",
78                 description: "",
79                 scheduled_for: "",
80                 category_id: null,
81                 status_id: null,
82                 is_recurring: false,
83                 recurrence_frequency: "daily",
84                 recurrence_interval: 1,
85                 recurrence_end_date: null,
86             });
87         }}
88         onSubmit={handleAddEvent}
89     />
90 )}
91
92 /* Event Details Modal */
93 {isEventModalOpen && selectedEvent && (
94     <EventDetailsModal
95         event={selectedEvent}
96         categories={categories}
97         statuses={statuses}
98         onClose={() => {
99             setIsEventModalOpen(false);
100             setSelectedEvent(null);

```

```

101         }}
102         onDelete={() => setDeleteModalOpen(true)}
103     />
104 }}
105
106 {/* Confirm Delete Modal */}
107 {deleteModalOpen && selectedEvent && (
108     <ConfirmDeleteModal
109         title={selectedEvent.title}
110         onCancel={() => setDeleteModalOpen(false)}
111         onConfirm={() => handleDeleteEvent(selectedEvent.id)}
112     />
113 )}
114
115 </div>
116 </div>
117 );

```

Listing 6.7: Definicja głównego widoku

- **Panel użytkownika** – zawiera odnośnik `User info`, który przekierowuje użytkownika do strony z informacjami o koncie.
- **Przycisk wylogowania** – umożliwia zakończenie sesji użytkownika.
- **Panel filtrowania wydarzeń** (`EventFiltersPanel` (rys. 7.2)) – pozwala filtrować wydarzenia według kategorii oraz statusu. Dodatkowo pokazuje liczbę aktualnie wyświetlanych wydarzeń względem wszystkich dostępnych.
- **Widok kalendarza** (`CalendarViewModel` (rys. 7.2)) – prezentuje przefiltrowane wydarzenia w układzie kalendarza. Obsługuje kliknięcie wybranego dnia oraz zmianę aktualnie widocznego miesiąca.
- **Modal wydarzeń danego dnia** (`DayEventsModal` (rys. 7.3)) – wyświetla listę wydarzeń przypisanych do wybranej daty. Z jego poziomu można przejść do dodawania nowego wydarzenia lub otworzyć szczegóły istniejącego wydarzenia.
- **Modal dodawania wydarzenia** (`AddEventModal` (rys. 7.6)) – zawiera formularz tworzenia nowego wydarzenia. Po zamknięciu formularz jest resetowany do wartości domyślnych.
- **Modal szczegółów wydarzenia** (`DayEventsModal` (rys. 7.4)) – prezentuje szczegółowe informacje o wybranym wydarzeniu, takie jak kategoria, status czy opis. Umożliwia także rozpoczęcie procesu usuwania wydarzenia.
- **Modal potwierdzenia usunięcia** (`ConfirmDeleteModal` (rys. 7.5)) – zabezpiecza użytkownika przed przypadkowym usunięciem wydarzenia, wymagając potwierdzenia tej operacji.

6.3.3. Definicja modalu

W celu zapewnienia spójnego wyglądu oraz zachowania jednolitej logiki działania okien modalnych, zaimplementowano komponent bazowy `Modal.tsx`, który jest wykorzystywany

przez pozostałe modale poprzez kompozycję (przekazywanie zawartości jako children). Takie podejście pozwala ograniczyć powtarzalność kodu, zwiększa jego czytelność oraz ułatwia dalszą rozbudowę aplikacji.

Zaimplementowany komponent odpowiada za wspólne elementy interfejsu, takie jak tło przyciemnienia, centrowanie modalu na ekranie, obsługa przewijania oraz opcjonalny przycisk zamknięcia. Dzięki temu poszczególne modale definiują jedynie swoją specyficzną zawartość, korzystając ze wspólnej struktury wizualnej i funkcjonalnej.

```

1 import { ModalProps } from "@types";
2
3 export default function Modal({ children, onClose, className }:
4   ModalProps) {
5   return (
6     <div className="fixed inset-0 z-[9999] flex items-center
7       justify-center bg-black/40">
8       <div
9         className={`relative pointer-events-auto w-[400px]
10          max-h-[80vh] overflow-y-auto rounded-xl bg-white p-4 shadow-xl
11          ${className} ?? ""}`>
12         >
13           {onClose && (
14             <button
15               type="button"
16               onClick={() => {
17                 onClose();
18               }}
19               className="absolute top-3 right-3 z-10 px-3 py-1
20               text-slate-500 bg-slate-100 rounded-lg hover:bg-slate-200
21               hover:text-slate-700 transition"
22             >
23               X
24             </button>
25           )}
26           {children}
27         </div>
28       </div>
29     );
30   }

```

Listing 6.8: Definicja modalu

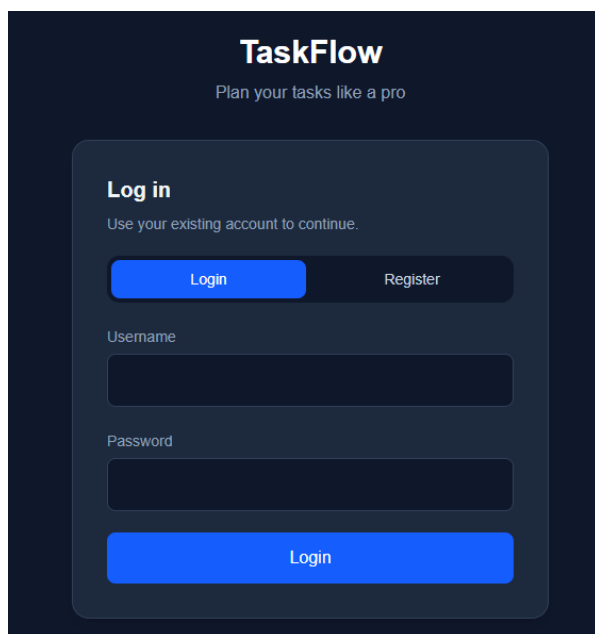
Rozdział 7

Wygląd aplikacji

Aplikacja składa się z 3 podstawowych widoków: **ekran logowania/rejestracji**, **ekran główny** oraz **ekran informacji o użytkowniku**. Ponadto, w widoku głównym istotną rolę odgrywają **modale**.

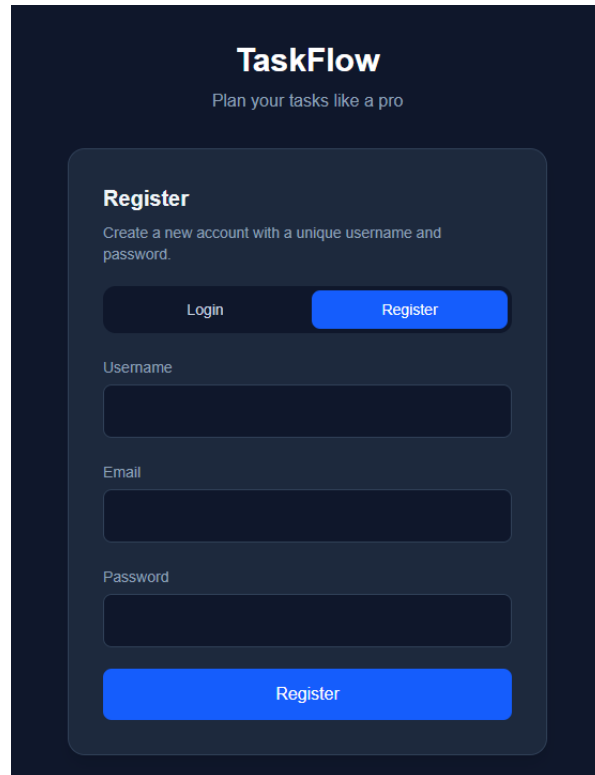
7.1. Widok logowania/rejestracji

Po otwarciu aplikacji ukazuje nam się panel, który pozwala na zalogowanie bądź zarejestrowanie się w aplikacji. Po pomyślnym zalogowaniu użytkownik zostaje przekierowany na główny widok. W przeciwnym wypadku ukazuje się komunikat o błędnym logowaniu.



The login screen for TaskFlow features a dark blue background. At the top, the 'TaskFlow' logo is displayed in white, with the tagline 'Plan your tasks like a pro' below it. A central light blue rounded rectangle contains the 'Log in' section. This section includes the text 'Use your existing account to continue.', a toggle switch with 'Login' selected, and input fields for 'Username' and 'Password'. A large blue 'Login' button is positioned at the bottom of the form.

(a) Logowanie



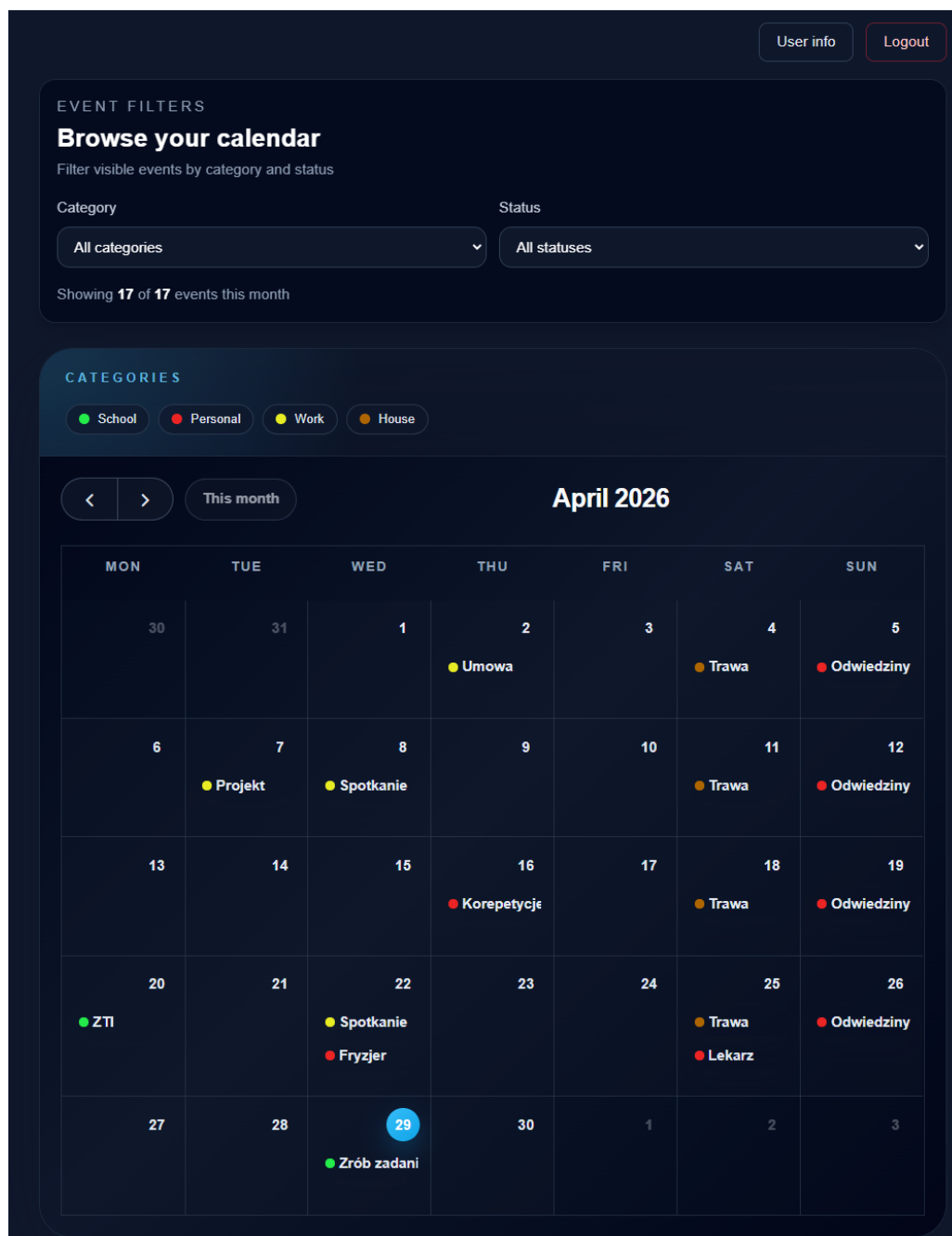
The registration screen for TaskFlow has a dark blue background. At the top, the 'TaskFlow' logo is shown in white, with the tagline 'Plan your tasks like a pro' underneath. A central light blue rounded rectangle contains the 'Register' section. This section includes the text 'Create a new account with a unique username and password.', a toggle switch with 'Register' selected, and input fields for 'Username', 'Email', and 'Password'. A large blue 'Register' button is located at the bottom of the form.

(b) Rejestracja

Rysunek 7.1: Ekran logowania i rejestracji

7.2. Widok główny

Po pomyślnym zalogowaniu użytkownik widzi kalendarz z umieszczonymi zadaniami/wydarzeniami. Na samej górze jest sekcja, za pomocą której użytkownik może filtrować wydarze-



Rysunek 7.2: Widok główny

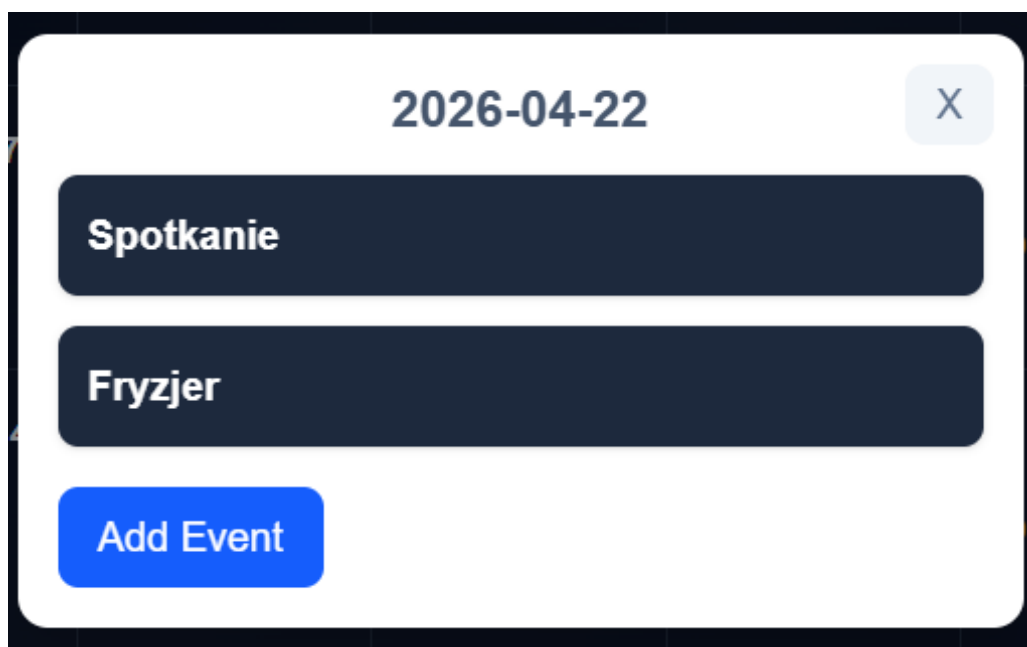
nia po kategorii i/lub statusie realizacji. W głównej sekcji tej strony jest kalendarz z umieszczonymi zadaniami w danym dniu, a powyżej legenda z kategoriami i ich kolorami, które widnieją w kalendarzu w celu lepszego UX.

7.3. Modale

Gdy użytkownik kliknie na sekcje odpowiedzialną za dany dzień ma możliwość podglądu na wydarzenia w danym dniu, ich edycji czy dodania nowego wydarzenia.

7.3.1. DayEventsModal

Po wybraniu konkretnego dnia który interesuje użytkownika otwiera się modal z wszystkimi wydarzeniami w danym dniu posortowanymi rosnąco według godziny. Tutaj użytkownik

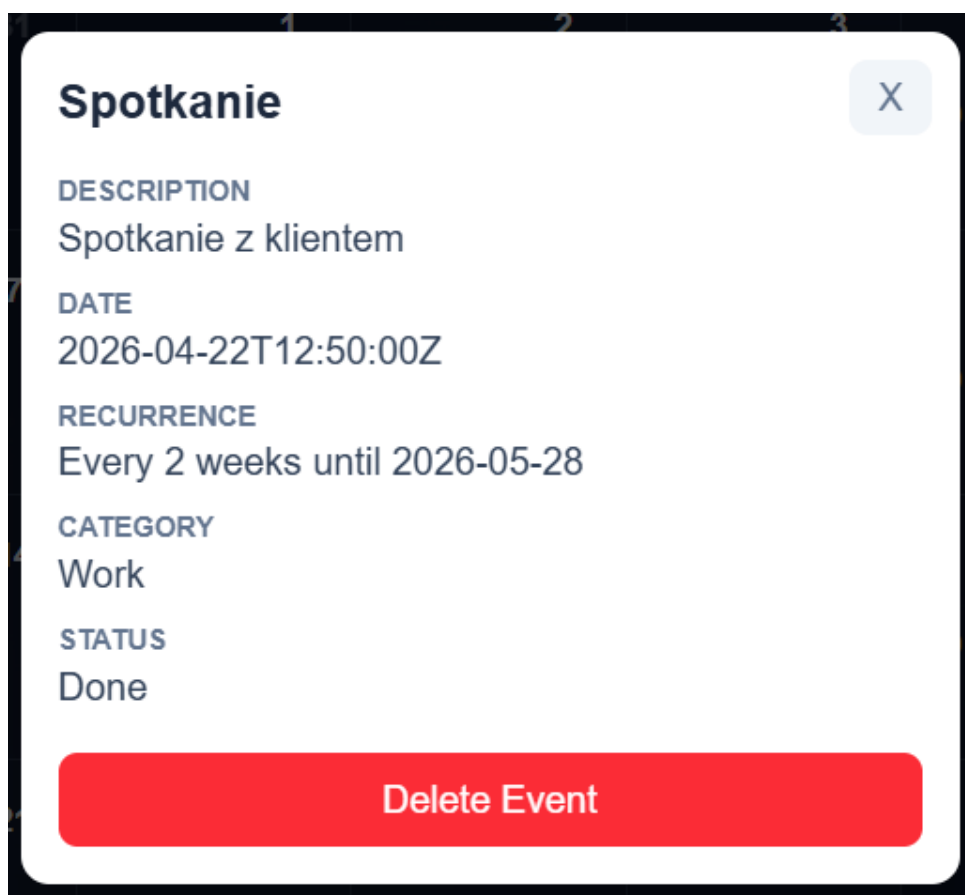


Rysunek 7.3: Modal z wszystkimi wydarzeniami w danym dniu

może kliknąć przycisk **Add Event** i stworzyć nowe wydarzenie bądź kliknąć na konkretne wydarzenie w celu uzyskania szczegółów.

7.3.2. EventDetailsModal

Po wybraniu danego wydarzenia ukazuje się następujący modal.

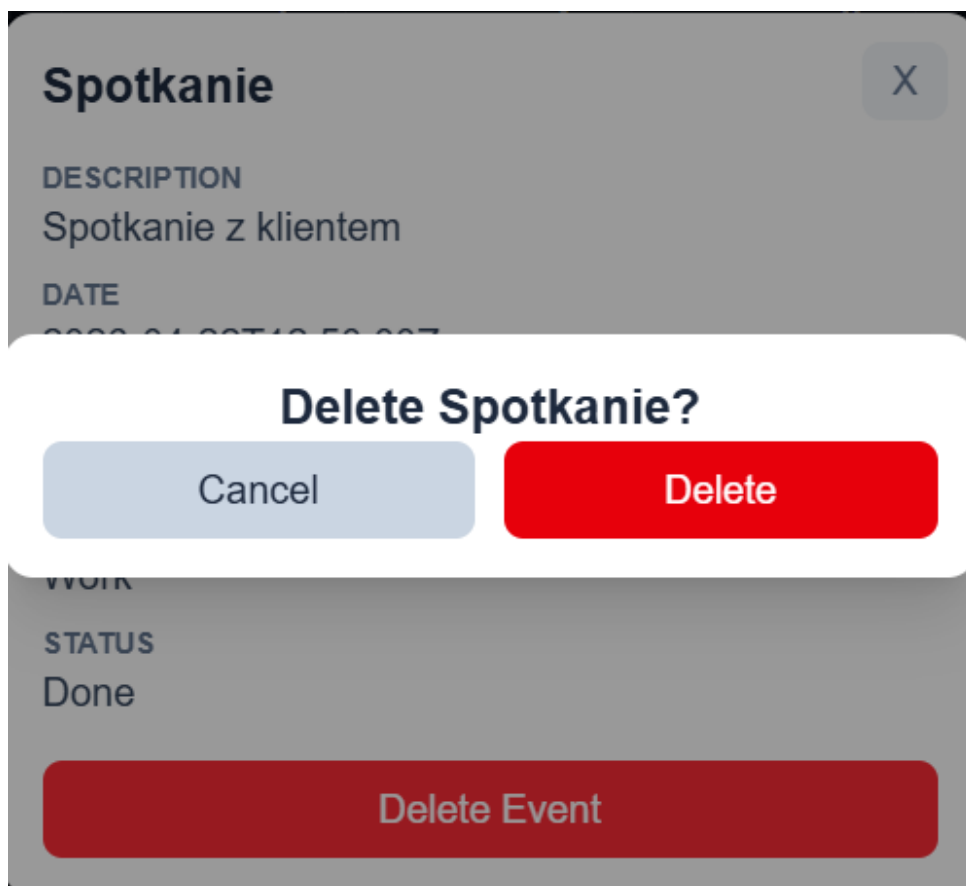


Rysunek 7.4: Modal ze szczegółami danego wydarzenia

Tutaj użytkownik może kliknąć na nazwę wydarzenia w celu jego edycji, bądź kliknąć **Delete Event** w celu jego usunięcia.

7.3.3. ConfirmDeleteModal

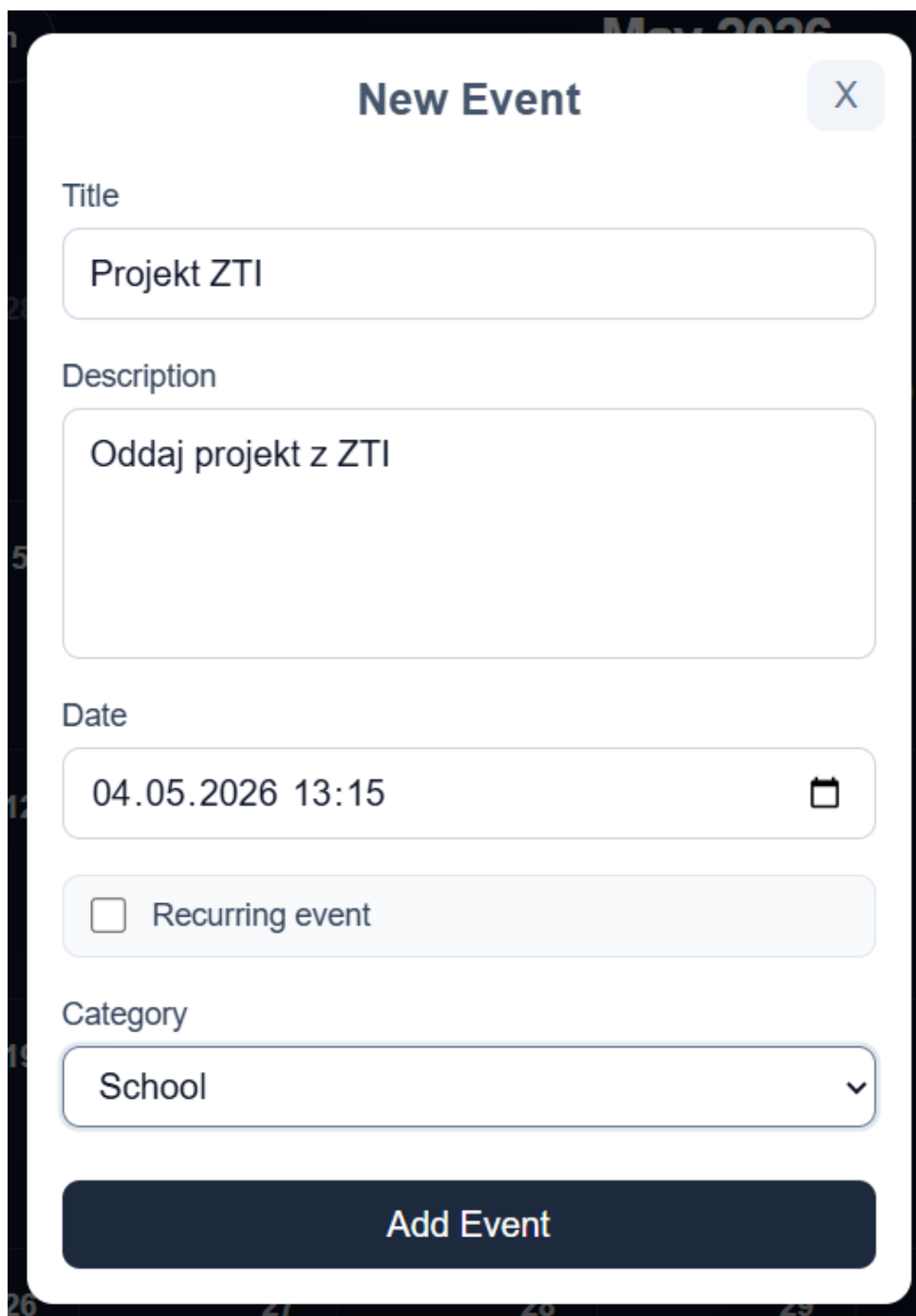
Aby uniknąć przypadkowego usunięcia wydarzenia, użytkownik jest pytany, czy na pewno chce usunąć dane wydarzenie.



Rysunek 7.5: Modal z pytaniem o potwierdzenie usunięcia wydarzenia

7.3.4. AddEventModal

Jeśli użytkownik chce dodać nowe wydarzenie ukazuje mu się następujący modal.



The image shows a 'New Event' modal form. At the top, the title 'New Event' is centered, with a close button (X) on the right. Below the title, there are four input fields: 'Title' with the text 'Projekt ZTI', 'Description' with the text 'Oddaj projekt z ZTI', 'Date' with the text '04.05.2026 13:15' and a calendar icon, and 'Category' with a dropdown menu showing 'School'. Below these fields is a checkbox labeled 'Recurring event'. At the bottom of the modal is a large blue button labeled 'Add Event'.

Rysunek 7.6: Modal z dodaniem nowego wydarzenia

7.3.5. Edycja istniejącego wydarzenia

Jeśli użytkownik chciałby edytować wydarzenie, które już istnieje zostaje przekierowany na adres URL strony tego konkretnego wydarzenia.

Edit event

Update the selected event details.

Title

ZTI

Description

Pokaż progres projektu

Date, status and category

Set when the event takes place and assign a status and a category.

Status

In Progress

Category

School

Date

20.04.2026 12:50

Cancel Save changes

Rysunek 7.7: Widok edycji wydarzenia

7.4. Informacje o użytkowniku

Użytkownik może również podejrzeć informacje o swoim profilu takie jak nazwa, email, utworzone kategorie i dostępne statusy. Może również dodawać nowe kategorie wraz z przypisaniem do nich koloru oraz nowe, customowe statusy.

ACCOUNT

User information

Manage your account details, event categories, and statuses

[Back to calendar](#) [Logout](#)

Your details

Username
Aleksander

Email
aleksander@example.com

Your categories

Here you can create custom categories to organize your events.

New category

e.g. Work, Personal, Studies

Category color

#2563eb

[Add category](#)

Saved categories

- School #24eb4b [Save color](#)
- Personal #eb2424 [Save color](#)
- Work #e7eb24 [Save color](#)
- House #ad6500 [Save color](#)

Rysunek 7.8: Widok profilu użytkownika

Rozdział 8

Testy

W ramach projektu przeprowadzono testy API z wykorzystaniem frameworka Django REST Framework. Testy zostały napisane zgodnie ze wzorcem **Arrange-Act-Assert**, co pozwala na czytelne rozdzielenie przygotowania danych, wykonania operacji oraz weryfikacji wyników. Testy obejmują kluczowe funkcjonalności systemu, takie jak obsługa wydarzeń cyklicznych, zarządzanie kategoriami oraz pobieranie statusów.

- `test_creating_recurring_event_creates_independent_event_rows` – sprawdza, czy utworzenie wydarzenia cyklicznego powoduje wygenerowanie osobnych rekordów dla każdego wystąpienia oraz przypisanie ich do jednej serii.
- `test_deleting_one_occurrence_deletes_whole_recurring_series` – weryfikuje, czy usunięcie jednego wystąpienia wydarzenia cyklicznego powoduje usunięcie całej serii wraz z powiązаныmi danymi.
- `test_category_list_returns_only_current_user_categories` – sprawdza, czy endpoint zwraca wyłącznie kategorie przypisane do aktualnie zalogowanego użytkownika.
- `test_creating_category_assigns_current_user_as_owner` – weryfikuje, czy podczas tworzenia kategorii jej właścicielem automatycznie zostaje aktualny użytkownik.
- `test_status_list_returns_available_statuses` – sprawdza poprawność działania endpointu zwracającego dostępne statusy wydarzeń.

Przykładowy test wygląda następująco.

```
1 class StatusesApiTests(APITestCase):
2     def setUp(self):
3         self.user = User.objects.create_user(
4             username="status-user",
5             email="status@example.com",
6             password="password123",
7         )
8         self.client.force_authenticate(user=self.user)
9
10    # Verifies that the statuses endpoint returns the available event
11    statuses.
12    def test_status_list_returns_available_statuses(self):
13        # Arrange
14        expected_status_names = ["Not Applicable", "To Do", "In Progress",
15                                "Done"]
```

```
14
15     # Act
16     response = self.client.get("/events/statuses/")
17
18     # Assert
19     self.assertEqual(response.status_code, http_status.HTTP_200_OK)
20     self.assertEqual(
21         [item["name"] for item in response.data],
22         expected_status_names,
23     )
```

Listing 8.1: Test zwracanych statusów wydarzeń

Test rozpoczyna się od utworzenia użytkownika testowego oraz wymuszenia jego uwierzytelnienia w metodzie `setUp`, co pozwala na wykonywanie zapytań jako zalogowany użytkownik. Następnie weryfikowana jest poprawność działania endpointu `/events/statuses/` poprzez sprawdzenie kodu odpowiedzi HTTP oraz zgodności zwróconej listy nazw statusów z oczekiwanym zestawem.

Rozdział 9

Wnioski

W ramach realizacji projektu udało się zaimplementować wszystkie zakładane funkcjonalności aplikacji, takie jak rejestracja i logowanie użytkowników, zarządzanie wydarzeniami (w tym wydarzeniami cyklicznymi) oraz ich filtrowanie według różnych kryteriów. Dodatkowo system został rozszerzony o funkcjonalności nieujęte pierwotnie w założeniach, m.in. możliwość definiowania własnych kategorii i statusów czy edycji wydarzeń.

Zastosowana architektura oparta na Django (backend), Next.js (frontend) oraz PostgreSQL (baza danych), uruchamiana w środowisku Docker, okazała się odpowiednia i umożliwiła stworzenie spójnej oraz skalowalnej aplikacji. Wysoka interaktywność interfejsu użytkownika została osiągnięta dzięki wykorzystaniu renderowania po stronie klienta (Client-Side Rendering), co pozytywnie wpłynęło na doświadczenie użytkownika.

Jednym z możliwych kierunków usprawnień jest lepsze wykorzystanie mechanizmów renderowania po stronie serwera (SSR), oferowanych przez Next.js. W obecnej implementacji, ze względu na dużą dynamikę i interaktywność aplikacji, zdecydowano się głównie na renderowanie po stronie klienta, jednak w przyszłości można rozważyć częściowe wykorzystanie SSR (np. dla mniej dynamicznych widoków), co mogłoby poprawić wydajność początkowego ładowania oraz SEO aplikacji.

Kolejnym naturalnym kierunkiem rozwoju projektu jest rozbudowa systemu o obsługę wydarzeń grupowych. Już na etapie projektowania bazy danych przewidziano taką możliwość (np. tabela `event_participants`), co znacząco ułatwi implementację tej funkcjonalności w przyszłości.

Warto również podkreślić, że jednym z głównych celów projektu było zdobycie praktycznej wiedzy z zakresu nowych technologii, takich jak Django oraz Next.js. Cel ten został w pełni zrealizowany, a zdobyte doświadczenie stanowi istotną wartość dodaną projektu i może zostać wykorzystane w dalszej pracy nad podobnymi systemami.